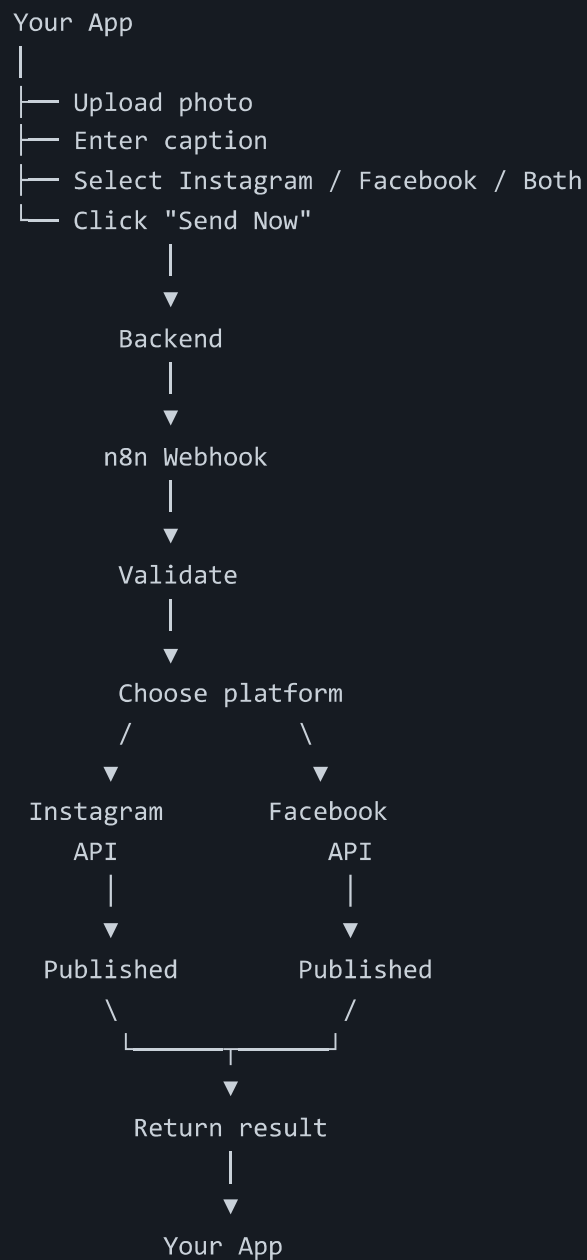


Yes. Let's build this **from absolute zero**, step by step. I'll separate what you configure in **Meta**, what you build in **your application**, and what you build in **n8n**.

Your uploaded guide's core model is exactly what we're going to use: **Trigger** → **Process** → **Action**.

What we are building

The final user experience will be:



PART 1 — Understand the 4 things you need

You asked an important question:

"Image I will upload right? How will it go as a path?"

There are **four separate things**:

1. Meta/Facebook setup

Needed so your application can communicate with Facebook.

2. Instagram setup

Needed so your application can communicate with Instagram.

3. n8n

This receives your application's request and executes the automation.

4. Image storage

Your uploaded image needs to be somewhere that the publishing API can access.

The image is **not simply a Windows path like**:

```
C:\Users\Sony\Desktop\photo.jpg
```

That path exists only on your computer.

Instead, your application uploads the image to storage and obtains a URL such as:

```
https://storage.yourapp.com/uploads/abc123.jpg
```

That URL is then passed through your system.

PART 2 — First setup: Facebook

Let's start with Meta/Facebook.

You need a **Facebook Page** for the Facebook publishing part.

Think:

```
Facebook Account
  |
  ▼
Facebook Page
  |
  ▼
Your Meta App
  |
  ▼
Your Application
```

Your application will eventually obtain authorization to publish to the Page.

Important distinction

Your target should generally be:

```
Facebook Page
```

rather than:

```
Personal Facebook profile
```

for this type of API publishing workflow.

PART 3 — Create your Meta Developer application

Go to Meta's developer platform and create an application.

You'll eventually have something conceptually like:

```
Meta Developer App
  |
  ├── App ID
  ├── App Secret
  └── Products / API configuration
```

Do not put the App Secret in your frontend.

Your secret belongs on your backend/server.

The n8n guide makes the same security point for API keys and access tokens: treat them as secrets and don't expose them in code, screenshots, exports, etc.

PART 4 — Instagram setup

For the Instagram side, don't think of it as:

```
Normal Instagram personal account
      ↓
    API
```

Instead, you're working with the Instagram API and an eligible **professional Instagram account**.

Conceptually:

```
Instagram Professional Account
      |
      ▼
    Meta ecosystem
      |
      ▼
    Meta App
      |
      ▼
    Your Application
```

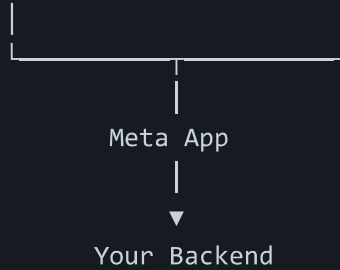
You'll connect the Instagram account through Meta's authentication system.

PART 5 — Connect Facebook Page and Instagram

This is important.

Your setup should look approximately like:

```
      META
      |
  _____|_____
  |                 |
Facebook Page    Instagram Professional
```



The exact account/product configuration and permissions depend on the Meta API flow you're using.

Don't start coding the publishing API yet.

First get the account connection working.

PART 6 — What permissions are involved?

This is where you were asking earlier:

"Can't we publish? We can give whatever permission it needs?"

Yes, **your application requests the permissions that Meta makes available and that your use case requires.**

But you cannot arbitrarily create permissions or bypass Meta's approval system.

There are two different situations:

Development/testing

You can test with accounts/roles that Meta allows for your app.

Real users

If your application will be used by customers, you'll need to configure the app appropriately and go through Meta's required review/approval process for permissions that require it.

So don't think:

Give every permission = automatically works

Think:

Your App
↓

```
Request required permission
↓
Meta checks eligibility/review requirements
↓
User authorizes
↓
Token
↓
API access
```

PART 7 — Now let's set up n8n

You asked:

```
"setup n8n in local or?"
```

For learning/development:

Yes, you can run n8n locally.

For example:

```
Your Windows PC
├── Your application
└── n8n
```

You can run n8n locally while developing.

But there is one important issue.

Your local n8n might have an address such as:

```
http://localhost:5678
```

Your application can access that on your computer.

But Meta/external services cannot normally call your `localhost` directly.

Therefore, during development you'll typically need a publicly reachable HTTPS URL/tunnel when an external service must reach your local n8n.

Later, for production:

Internet
↓
Your server
↓
n8n

PART 8 — Start n8n

Once n8n is running, open its interface.

You'll create a workflow.

Start with:

Webhook

Your first workflow is:

Webhook
↓
Respond to Webhook

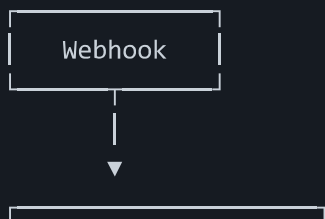
Don't add Instagram yet.

First prove that your application can communicate with n8n.

This follows the guide's recommendation to build incrementally and inspect the real data at each step.

PART 9 — Create your n8n Webhook

Your n8n workflow:



Respond to Webhook

The Webhook gives your application an endpoint.

Conceptually:

```
POST https://your-n8n-domain/webhook/social-post
```

Your application will call this.

The guide describes Webhook exactly for this purpose: another application/service sends data to n8n, and that incoming payload becomes workflow data.

PART 10 — Now your application

Your frontend can be React, Next.js, Angular, etc.

Let's imagine React.

Your page:

Create Post

[Upload Image]

Image Preview

Caption:

[_____]

Platform:

- ☐ Instagram
- ☐ Facebook
- ☐ Both

[SEND NOW]

The user chooses:

Image:

product.jpg

Caption:

Our new product is available!

Platform:

Instagram

PART 11 — The image question

This is VERY important.

Suppose your user selects:

```
C:\Users\Sony\Pictures\product.jpg
```

You should **not** send that Windows path to Meta.

Why?

Because:

```
C:\Users\Sony\Pictures\product.jpg
```

only exists on the user's computer.

Your server/n8n/Meta cannot use that local path.

Instead:

```
graph TD
    A[User's computer] -- upload --> B[Your storage]
    B --> C[https://storage.yourapp.com/images/abc123.jpg]
```

Now you have a URL.

PART 12 — Where should the image be stored?

You have several choices.

For development, you could use object storage such as:

```
Cloud storage
  ↓
Image URL
```

For production, use proper object storage/CDN infrastructure.

The important concept is:

```
UPLOAD
  ↓
STORAGE
  ↓
PUBLIC/ACCESSIBLE IMAGE URL
  ↓
n8n
  ↓
Meta API
```

The exact storage provider is your choice.

PART 13 — Your backend

I strongly recommend:

```
Frontend
  ↓
Backend
  ↓
n8n
```

rather than:

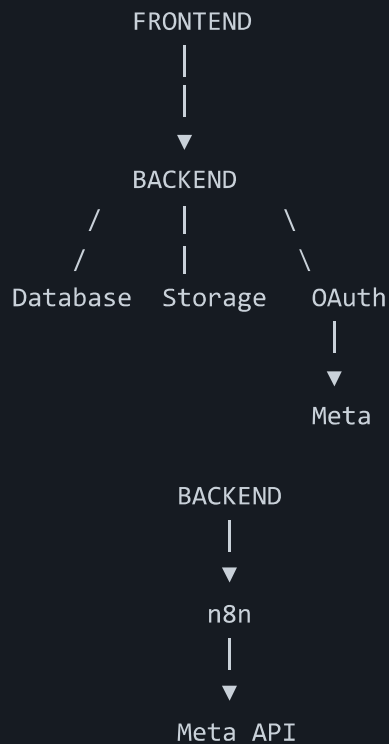
```
Frontend
  ↓
n8n directly
```

Why?

Because your backend should handle things like:

- user authentication
- social-account connections
- access tokens
- database
- image upload
- post records
- security
- authorization

Architecture:



PART 14 — What happens when user uploads image?

Let's follow one image completely.

User selects:

product.jpg

Frontend:

```
product.jpg
  ↓
POST /upload
  ↓
Backend
```

Backend uploads it to storage.

Storage returns:

```
https://storage.example.com/images/a8f31.jpg
```

Backend saves:

```
image_url =
https://storage.example.com/images/a8f31.jpg
```

PART 15 — User clicks Send Now

Now your frontend sends:

```
{
  "imageUrl": "https://storage.example.com/images/a8f31.jpg",
  "caption": "Our new product is available!",
  "platform": "instagram"
}
```

Your backend receives this.

Then backend sends it to n8n.

PART 16 — n8n receives the JSON

Webhook gets:

```
{
  "userId": "123",
  "imageUrl": "https://storage.example.com/images/a8f31.jpg",
  "caption": "Our new product is available!",
```

```
"platform": "instagram"
}
```

Now you have your:

TRIGGER

Webhook

PART 17 — n8n Process

Now add:

```
Webhook
  ↓
Edit Fields
  ↓
IF / Switch
```

For example:

```
Webhook
  ↓
Edit Fields
  ↓
Switch
```

The Switch reads:

platform

If:

instagram

go to Instagram.

If:

facebook

go to Facebook.

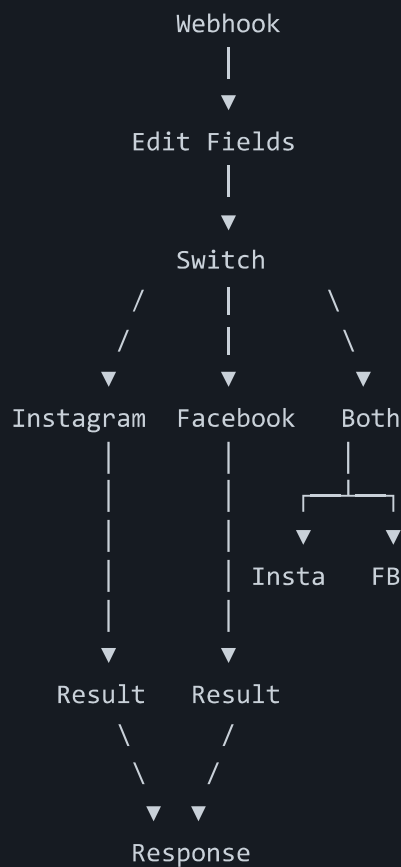
If:

both

go to both.

The guide specifically describes Switch as the node for routing data into multiple possible branches.

PART 18 — Your n8n workflow becomes



PART 19 — Instagram action

The Instagram branch will use Meta's API.

Conceptually:

```
Instagram branch
  ↓
Create media/container
  ↓
Wait/check processing
  ↓
Publish media
  ↓
Instagram post ID
```

The exact API request configuration must follow Meta's current API documentation and the authentication flow you configure.

n8n's **HTTP Request** node is the piece that lets you communicate with an API. Your guide describes the four things you'll need to understand: endpoint, method, authentication, and parameters/body.

PART 20 — Facebook action

Facebook branch:

```
Facebook branch
  ↓
HTTP Request
  ↓
Facebook Graph API
  ↓
Facebook Page
  ↓
Post ID
```

Again, n8n is not magically publishing itself.

It's doing:

```
n8n
  ↓
HTTP Request
  ↓
Meta API
  ↓
Facebook
```

PART 21 — Credentials

This is one of the most important parts.

You may have:

```
Meta App ID
Meta App Secret
Access Token
Facebook Page ID
Instagram Account ID
```

These are **not frontend variables**.

Don't do this:

```
const ACCESS_TOKEN = "xxxxx";
```

inside React.

Instead:

```
Frontend
  ↓
Backend
  ↓
Secure credential/token storage
  ↓
n8n
  ↓
Meta
```

Your guide specifically says API keys and access tokens should be treated as secrets.

PART 22 — What exactly happens during Send Now?

Let's follow one real request.

User:

Photo:
burger.jpg

Caption:
Try our new burger!

Platform:
Instagram

Step 1

Frontend uploads:

burger.jpg

↓

Step 2

Storage gives:

<https://storage.com/abc/burger.jpg>

↓

Step 3

Frontend/backend creates:

```
{  
  "imageUrl": "https://storage.com/abc/burger.jpg",  
  "caption": "Try our new burger!",  
  "platform": "instagram"  
}
```

↓

Step 4

Backend sends this to n8n.

↓

Step 5

Webhook receives it.

↓

Step 6

n8n validates:

```
image exists?      YES
caption exists?    YES
platform valid?    YES
```

↓

Step 7

Switch:

```
instagram
```

↓

Step 8

Instagram API:

```
Create media
```

↓

Step 9

Instagram processes the media.

↓

Step 10

n8n checks status.

↓

Step 11

When ready:

Publish

↓

Step 12

Instagram returns the post ID.

↓

Step 13

n8n returns:

```
{
  "success": true,
  "platform": "instagram",
  "postId": "123456"
}
```

↓

Step 14

Backend saves:

```
posts
-----
user_id
platform
image_url
caption
post_id
status
created_at
```

↓

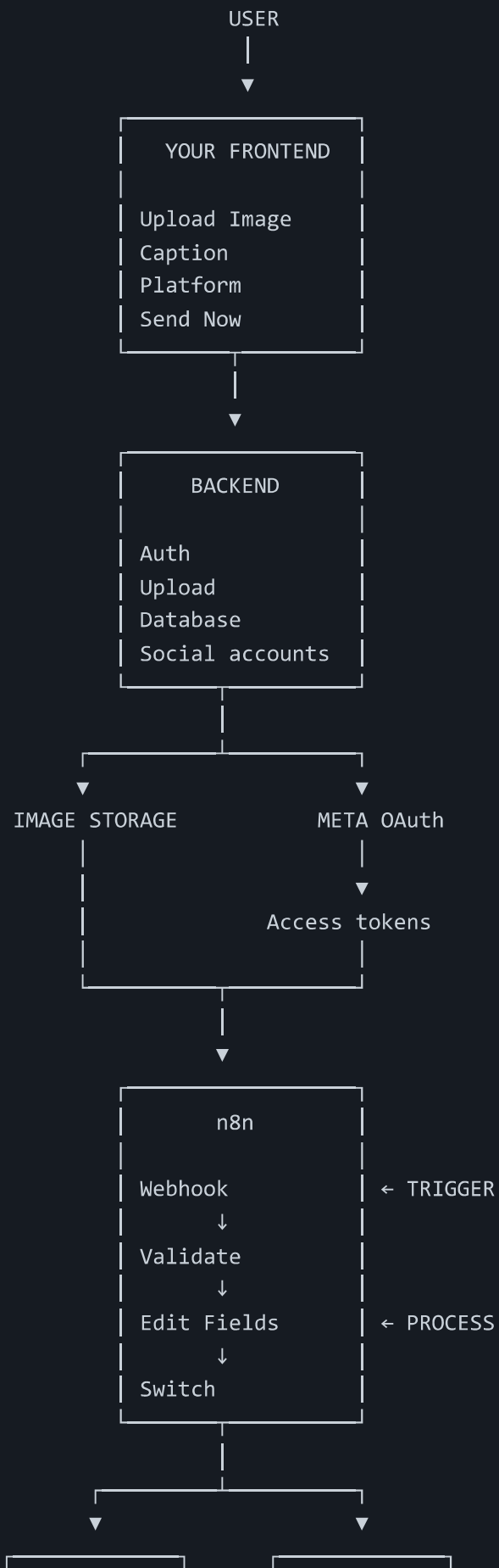
Step 15

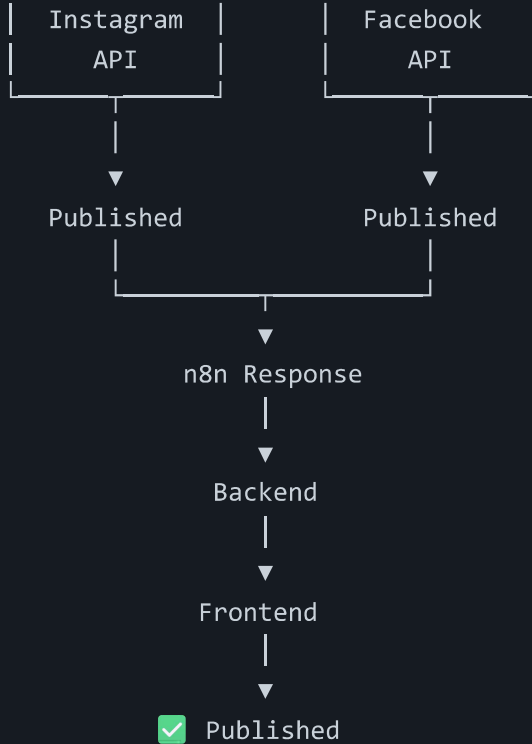
Frontend displays:

✓ Published successfully!

PART 23 — Your complete system

Now put everything together:





PART 24 — What you need to build, in order

Don't try to build everything at once.

Build it in these stages:

Phase 1 — Meta

- ☐ Facebook account
- ☐ Facebook Page
- ☐ Instagram Professional account
- ☐ Connect Instagram/Page as required
- ☐ Create Meta Developer App
- ☐ Configure required products
- ☐ Configure OAuth
- ☐ Configure required permissions

Phase 2 — n8n

- ☐ Install n8n
- ☐ Start n8n
- ☐ Create Webhook
- ☐ Test webhook
- ☐ Receive JSON
- ☐ Learn expressions

- ☐ Add validation
- ☐ Add Switch

This directly follows your guide's progression through Webhooks, JSON, expressions, data shaping, and branching.

Phase 3 — Image

- ☐ Create upload endpoint
- ☐ Upload image
- ☐ Store image
- ☐ Get image URL
- ☐ Test that URL is accessible

Phase 4 — Your application

- ☐ Login
- ☐ Upload UI
- ☐ Caption UI
- ☐ Platform selector
- ☐ Send Now button

Phase 5 — Connect app → n8n

- ☐ Frontend
 - ↓
- ☐ Backend
 - ↓
- ☐ n8n Webhook

Phase 6 — Instagram

- ☐ Authentication
- ☐ Create media
- ☐ Check processing
- ☐ Publish
- ☐ Handle response

Phase 7 — Facebook

- ☐ Authentication
- ☐ Page access

- ☐ Publish photo
- ☐ Handle response

Phase 8 — Production

- ☐ Error handling
- ☐ Logging
- ☐ Token management
- ☐ Retry handling
- ☐ Security
- ☐ Meta App Review where required
- ☐ HTTPS
- ☐ Production n8n

The guide also emphasizes validation, error handling, logging, credentials, and reliable business rules as production concerns.

The most important thing: DON'T build all of this today

We should build it one checkpoint at a time.

I would start with this exact sequence:

```
STEP 1
Create Facebook Page
    ↓
STEP 2
Prepare Instagram Professional account
    ↓
STEP 3
Create Meta Developer App
    ↓
STEP 4
Configure Meta authentication
    ↓
STEP 5
Install n8n locally
    ↓
STEP 6
Create n8n Webhook
    ↓
STEP 7
Send test JSON from your app
    ↓
```

```
STEP 8
Make image upload work
    ↓
STEP 9
Connect Meta API
    ↓
STEP 10
Test Instagram publishing
    ↓
STEP 11
Test Facebook publishing
    ↓
STEP 12
Connect everything together
```

For the image specifically: your user uploads a file → **your backend/storage saves it** → you get an accessible image URL → send that URL through n8n → Meta uses the image URL for publishing. You should **not** send `C:\...` or a browser-local file path to n8n/Meta.

If we build this together, **start with Step 1: Facebook Page + Instagram Professional account + Meta Developer App**. After that, we can move to the n8n local installation and create the first Webhook.